bond_pricing

November 5, 2026

1 Corporate bond pricing in OSEM

This notebook walks through how OSEM treats a corporate bond position: from raw input data, to projected cash flows, to a calibrated market-consistent price. Each corporate bond is defined by its characteristic, the market conditions such as spread and price.

On a very high level:

1) Each corporate bond is saved into a CorpBond object.
2) A portfolio of CorpBond objects is then saved into the BondPortfolio.
3) Based on the BondPortfolio and market conditions, the cash flows resulting from each bond are reconstructed into a matrix where the cash flow dates are in the columns and individual bonds in rows.
4) Based on the cash flows and current market conditions, the z-spread is calculated using the bisection algorithm to return the current market price.

1.1 Table of contents

1. Methodology
   - Notation
   - Step 1 — Cash flow dates
   - Step 2 — Date fractions
   - Step 3 — Cash flow amounts
   - Step 4 — Pricing off the risk-free curve
   - Step 5 — Calibrating the z-spread
   - Step 6 — Calibrating the whole portfolio
   - Step 7 — Use in the full OSEM run
   - Generating new bonds
2. Setup
   - Environment and imports
   - Run parameters
3. The bond portfolio
   - Loading the bonds
   - Which fields drive the price
   - Sector spread input
4. Steps 1 and 3 in numbers: cash flow dates and amounts
   - Coupon and notional flows
   - Cash flow matrices
5. Step 4 in numbers: pricing off the risk-free curve

- The risk-free curve
- Pricing state at the modelling date
- Risk-free price: the gap the z-spread has to close
- Price at the input z-spread

6. Step 5 in numbers: calibrating one bond
7. Step 6 in numbers: calibrating the whole portfolio
8. Appendix: input data
    - Parameters.csv
    - EIOPA RFR files
    - Corporate_Bond_Portfolio.csv
    - Sector_Spread.csv

1.2 1. Methodology

How OSEM turns a bond description into a price, step by step. No code in this section; the sections that follow run each step on the input portfolio.

1.2.1 Notation

This table defines the mathematical notation that will be used through the rest of the workbook

Symbol	Meaning
MD	modelling date
VD	valuation date, the date cash flows are discounted to ($VD = MD$ at $t = 0$)
E	end of the modelling window (`settings.end_date`)
I	issue date
M	maturity date
L	last cash flow date, $\min(E, M)$
N	notional amount
c	coupon rate, paid on **each** coupon date
f	coupon frequency (payments/year)
MV	market value (price) at the modelling date
z	bond-specific z-spread
$y(t)$	risk-free spot yield for maturity t, from the calibrated EIOPA curve

A note on f. The frequency sets only the *spacing* of the coupon dates, through `relativedelta(months=12 // f)`. It does **not** scale the coupon amount: c is the rate paid on each coupon date, not an annualised rate, so a bond with $c = 0.02$ and quarterly frequency pays 2% of the notional four times a year. A bond quoted with an annual coupon c_{ann} paid f times a year is entered in `Corporate_Bond_Portfolio.csv` with `Coupon_Rate` $= c_{ann}/f$.

1.2.2 Step 1 — Cash flow dates

Every bond produces two kinds of cash flow: coupons and the return of notional.

A bond still alive when the modelling window closes is redeemed there at par, so both kinds of cash flow stop at

$$L = \min(E, M)$$

Notional repayment date:

$$t_M^d = L$$

Coupon dates:

$$I < t_1^d < t_2^d < \cdots < t_k^d \leq L$$

The coupon grid is anchored on the issue date and stepped by $12/f$ months. Payments falling before MD are dropped, since only future cash flows are priced.

In numbers: section 4.

1.2.3 Step 2 — Date fractions relative to the valuation date

Only cash flows after the valuation date matter. Each remaining cash flow date is converted to a year fraction from it:

$$t_i = \frac{t_i^d - VD}{365.25}$$

At the modelling date $VD = MD$, which is the case this notebook demonstrates. In a full run the valuation date moves forward with every period and the fractions are re-measured from the new date.

This is what lets the discounting step treat every cash flow generically as "an amount, this many years from now."

Step 2 runs inside `price_bond` *and has no table of its own: the year fractions are the maturities at which the discount factors in section 5 are evaluated.*

1.2.4 Step 3 — Cash flow amounts

Notional cash flow (paid once, at maturity):

$$cf_M = N$$

Coupon cash flows (paid at each coupon date):

$$cf_i = N \cdot c$$

In numbers: section 4.

1.2.5 Step 4 — Pricing off the risk-free curve

Before any bond-specific adjustment, each cash flow can be discounted purely with the risk-free spot curve calibrated in `CurvesClass`:

$$MV_{\text{rf}} = \sum_{i=1}^{k} \frac{cf_i}{(1 + y(t_i))^{t_i}} + \frac{cf_M}{(1 + y(t_M))^{t_M}}$$

This is *not* the bond's actual market price — it ignores credit/liquidity risk entirely. The gap between it and the real `Market_Price` from the input file is exactly what the z-spread in Step 5 is calibrated to close.

In numbers: section 5.

1.2.6 Step 5 — Calibrating the z-spread to match the market price

OSEM finds a single constant z added to the risk-free yield at every maturity such that the discounted cash flows reproduce the bond's observed market price exactly:

$$MV = \sum_{i=1}^{k} \frac{cf_i}{(1 + y(t_i) + z)^{t_i}} + \frac{cf_M}{(1 + y(t_M) + z)^{t_M}}$$

There is no closed-form solution for z, so OSEM finds it numerically with a **bisection search** (`bisection_spread`) over a bracket $[z_{\text{start}}, z_{\text{end}}]$, which `calibrate_bond_portfolio` fixes at $[-0.2, 0.2]$.

The bracket has to contain a solution. The price falls monotonically in z, so a root exists only if MV lies between the prices at the two ends of the bracket. `bisection_spread` checks this before iterating and raises a `ValueError` naming the bond when it fails. Without that check the search never sees a sign change, walks z_{start} all the way up to z_{end}, and returns the bracket bound as though it were a calibrated spread. A market price outside the bracket usually means either that the bracket is too narrow for a genuinely distressed bond, or that `Coupon_Rate` was entered as an annualised rate (see the note on f above).

What `precision` measures. Once bracketed, the search stops when the bracket is narrower than `precision`, that is $(z_{\text{end}} - z_{\text{start}})/2 <$ `precision`. This is a tolerance on the *spread*, not on the price. `calibrate_bond_portfolio` hardcodes `precision` $= 10^{-8}$, which leaves a residual of order 10^{-6} in price on the bonds in this portfolio. The two bracket endpoints are the exception: each is returned straight away if it already prices within `precision` of MV.

In numbers: section 6.

1.2.7 Step 6 — Calibrating the whole portfolio

`calibrate_bond_portfolio` simply repeats Step 5's bisection for every bond in the portfolio, storing one z per asset in `zspread_df`.

In numbers: section 7.

1.2.8 Step 7 — How this is used in the full OSEM run

This notebook prices bonds once, at $t = 0$. In a full run (`main.py`):

1. The portfolio's z-spreads are calibrated **once**, at the modelling date (`calibrate_bond_portfolio`, called in `main.py` before the main loop starts).
2. In every subsequent period of the simulation, `price_bond_portfolio` is called again with that *same, fixed* z-spread but the *period's* point on the projected yield curve (the bond repricing step of the main loop) — so the bond's price moves only because the risk-free curve and remaining cash flows change, not because its credit spread is re-estimated.

This notebook only demonstrates the fixed-income leg in isolation; the full run also handles equities, cash, liabilities/unit-linked policies, and portfolio rebalancing each period.

1.2.9 Generating new bonds (not yet implemented)

OSEM's documentation also describes a process for generating *new* corporate bonds at each future period (to replace maturing debt), calibrating their coupon so they price at par:

$$1 = \sum_{i=1}^{k} \frac{dy}{(1 + y(t_i) + c + s + ss)^{t_i}} + \frac{1}{(1 + y(t_M) + c + s + ss)^{t_M}}$$

This is not yet implemented in `BondClasses.py` / this notebook — see `Documentation/OSEM_Documentation_draft.ipynb` (cells 84–99) for the full methodology.

1.3 2. Setup

Prepares the environment and loads the run parameters. Nothing in this section is specific to bonds.

1.3.1 Environment and imports

Locate the repository root (the folder holding `ALM.ini`), make the `osem` package importable from `src/`, and import the parts of OSEM this notebook uses.

[1]:
```
import os
import sys
import numpy as np
import pandas as pd
import datetime as dt
from pathlib import Path

# Walk up to the repository root, the folder holding ALM.ini
base_folder = next(path for path in [Path.cwd(), *Path.cwd().parents] if (path /
 ↪ "ALM.ini").is_file())
sys.path.insert(0, str(base_folder / "src"))
os.chdir(base_folder)
base_folder = os.getcwd()  # Get current working directory

from osem.CurvesClass import Curves
```

```
from osem.ImportData import import_SWEiopa, get_corporate_bonds,␣
 ↪get_configuration, get_settings
from osem.BondClasses import *
from osem.ConfigurationClass import Configuration
from osem.MainLoop import create_cashflow_dataframe
```

1.3.2 Run parameters

The notebooks read `notebooks/data/notebooks.ini`, a copy of `ALM.ini` that names a small, fixed input set in `notebooks/data/Input`, so the results below do not change with the model's live `Input` folder. It gives the location of every input file:

[2]:
```
conf: Configuration
conf = get_configuration(os.path.join(base_folder, "notebooks", "data",␣
 ↪"notebooks.ini"), os)
```

[3]:
```
parameters_file = conf.input_parameters
cash_portfolio_file = conf.input_cash_portfolio
corporate_bond_portfolio_file = conf.input_corporate_bond_portfolio
```

[4]:
```
paramfile = pd.read_csv(parameters_file).set_index("Parameter")
```

The parameter file is:

---

[5]:
```
display(paramfile)
```

Parameter	Value
EIOPA_param_file	notebooks/data/Input/Param_no_VA.csv
EIOPA_curves_file	notebooks/data/Input/Curves_no_VA.csv
country	Slovenia
n_proj_years	50
Precision	1E-10
Tau	0.0001
compounding	-1
Modelling_Date	29/04/2023
liability_mode	cashflow
random_seed	42

The settings object holds data about file locations, information about the run settings and model parameters such as modelling date.

[6]:
```
settings = get_settings(parameters_file)
```

The run that the rest of the notebook prices against:

---

[7]:
```
run_settings = pd.DataFrame(
    # Rendered as strings: in a mixed-type column pandas applies one float␣
 ↪format to every
    # numeric value, which would show a precision of 1e-10 as 0.0
    {"Value": [str(settings.modelling_date),
               str(settings.end_date),
               str(settings.n_proj_years),
               settings.country,
               f"{settings.precision:g}",
               f"{settings.tau:g}"]},
    index=["Modelling date",
           "End of modelling window",
           "Projection years",
           "Country",
           "Precision (curve calibration)",
           "Tau"],
)

display(run_settings)
```

	Value
Modelling date	2023-04-29
End of modelling window	2073-04-29
Projection years	50
Country	Slovenia
Precision (curve calibration)	1e-10
Tau	0.0001

1.4 3. The bond portfolio

1.4.1 Loading the bonds

A Python generator reads the bond portfolio file and encodes it into a dictionary based on the asset id. Each asset id contains a CorpBond object describing a single fixed income position.

[8]:
```
bond_input_generator = get_corporate_bonds(corporate_bond_portfolio_file)
bond_input = {corp_bond.asset_id: corp_bond for corp_bond in␣
 ↪bond_input_generator}
```

As an example, a single corporate bond in a CorpBond would look something like:

---

[9]:
```
display(bond_input["BE6285455497"])
```

```
CorpBond(asset_id='BE6285455497', nace='C11.0.5', issuer=None,␣
 ↪issue_date=datetime.date(2016, 3, 17), maturity_date=datetime.date(2028, 3,␣
 ↪17), coupon_rate=0.02, notional_amount=100.0, spread_country=0.0,␣
 ↪spread_sector=0.0, zspread=0.0052, spread_stress=0.0, frequency=<Frequency.
 ↪ANNUAL: 1>, recovery_rate=0.4, default_probability=0.00077, units=100.0,␣
 ↪market_price=93.4649)
```

`BondPortfolio` wraps a dictionary of these objects, keyed by asset id, and provides the portfolio-level methods used in the rest of the notebook.

[10]:
```
bond_portfolio = BondPortfolio(bond_input)
```

The whole fixed income portfolio at a glance. Note that `Annual coupon` is derived, not an input: it is `Coupon rate` times `Frequency`, because the coupon rate is the rate paid on each coupon date rather than an annualised one.

---

[11]:
```
portfolio_overview = pd.DataFrame(
    [{"Maturity": bond.maturity_date,
      "Notional": bond.notional_amount,
      "Coupon rate": bond.coupon_rate,
      "Frequency": int(bond.frequency),
      "Annual coupon": bond.coupon_rate * int(bond.frequency),
      "Units": bond.units,
      "Market price": bond.market_price}
     for bond in bond_portfolio.bonds.values()],
    index=list(bond_portfolio.bonds),
)

display(portfolio_overview)
```

```
               Maturity  Notional  Coupon rate  Frequency  Annual coupon  \
BE6285455497  2028-03-17     100.0      0.02000          1        0.02000
FR001400CFW8  2029-09-01     100.0      0.03625          1        0.03625
FR0013324357  2030-03-21     100.0      0.01375          1        0.01375
FR0000471930  2033-01-28     100.0      0.08125          1        0.08125
FR001400DNF5  2042-06-30     100.0      0.05375          1        0.05375

              Units  Market price
BE6285455497  100.0       93.4649
FR001400CFW8  100.0       99.4582
FR0013324357  100.0       86.4098
FR0000471930  100.0      132.6311
FR001400DNF5  100.0      108.2142
```

1.4.2 Which fields drive the price

Only seven of the `CorpBond` fields enter the price. The rest are loaded from the portfolio file but play no part in it; credit risk is carried by the z-spread alone.

`CorpBond` field	CSV column	Used in pricing	Role
`asset_id`	`Asset_ID`	–	Key of the portfolio dictionary and row index of every DataFrame
`issue_date`	`Issue_Date`	Yes	Anchors the coupon grid (I, Step 1)
`maturity_date`	`Maturity_Date`	Yes	Notional repayment date (M, Step 1)
`frequency`	`Frequency`	Yes	Spacing of the coupon dates, $12/f$ months (f, Step 1)
`notional_amount`	`Notional_Amount`	Yes	Notional repaid at L (N, Step 3)
`coupon_rate`	`Coupon_Rate`	Yes	Coupon paid on each coupon date, as a fraction of N (c, Step 3)
`market_price`	`Market_Price`	Yes	Calibration target (MV, Step 5)
`zspread`	`Z_Spread`	Starting value only	Seeds `zspread_df`; overwritten by the calibrated z-spread (Step 6)
`units`	`Units`	No	Number of bonds held; scales the cash flows in the main loop, not the per-unit price
`nace`	`NACE`	No	Sector code; would link the bond to `Sector_Spread.csv`, not used in the POC
`spread_country`, `spread_sector`, `spread_stress`	`Spread_Country`, `Spread_Sector`, `Spread_Stress`	No	Loaded but not used in bond pricing
`recovery_rate`, `default_probability`	`Recovery_Rate`, `Default_Probability`	No	Loaded but not used; there is no default model yet
`issuer`	–	No	Not read from the CSV; always `None`

1.4.3 Sector spread input

The sector spread file lists the NACE classification and the spread components that go with it. In this proof of concept the spread is taken directly from the bond input file, so `spreadfile` is loaded here for reference only.

[12]:
```
spreadfile = pd.read_csv(conf.input_spread)
spreadfile.index = spreadfile["NACE"]
del spreadfile["NACE"]
```

1.5 4. Steps 1 and 3 in numbers: cash flow dates and amounts

This section builds the coupon and notional schedules of Step 1 and the amounts of Step 3 for every bond. Step 2, the conversion to year fractions, happens inside `price_bond` when the flows are discounted in the next section.

1.5.1 Coupon and notional flows

The coupon payments and the return of the notional are simulated separately.

`create_coupon_flows` returns the coupon dates and amounts of every bond, as a dictionary keyed by asset id:

[13]:
```
coupon_flows = bond_portfolio.create_coupon_flows(settings.modelling_date,␣
 ↪settings.end_date)
```

`create_maturity_flows` returns the notional repayments in the same shape:

[14]:
```
notional_flows = bond_portfolio.create_maturity_flows(terminal_date=settings.
 ↪end_date)
```

A compact view of the same two dictionaries, one row per bond. The last coupon never falls after the date the notional is repaid, which is $L = \min(E, M)$ from Step 1:

---

[15]:
```
coupon_schedule = pd.DataFrame({
    "Coupons": {aid: len(flows) for aid, flows in coupon_flows.items()},
    "Coupon amount": {aid: next(iter(flows.values())) for aid, flows in␣
 ↪coupon_flows.items()},
    "First coupon": {aid: min(flows) for aid, flows in coupon_flows.items()},
    "Last coupon": {aid: max(flows) for aid, flows in coupon_flows.items()},
    "Notional repaid": {aid: min(flows) for aid, flows in notional_flows.
 ↪items()},
})

display(coupon_schedule)
```

	Coupons	Coupon amount	First coupon	Last coupon	Notional repaid
BE6285455497	5	2.000	2024-03-17	2028-03-17	2028-03-17
FR001400CFW8	7	3.625	2023-09-01	2029-09-01	2029-09-01

```
FR0013324357        7          1.375   2024-03-21  2030-03-21      2030-03-21
FR0000471930       10          8.125   2024-01-28  2033-01-28      2033-01-28
FR001400DNF5       20          5.375   2023-06-30  2042-06-30      2042-06-30
```

1.5.2 Cash flow matrices

All cash flows can be represented in a matrix with all possible cash flow dates as columns and all bonds as rows. The non-zero entries are the cash flow per unit of the bond at that date. The columns are the unique payment dates across the portfolio, from `unique_dates_profile`, computed separately for coupons and notional repayments:

[16]:
```
unique_list = bond_portfolio.unique_dates_profile(coupon_flows)
```

[17]:
```
unique_terminal_list = bond_portfolio.unique_dates_profile(notional_flows)
```

A dataframe of cash flows and notional amount payments is created:

[18]:
```
# Dataframe with bond  coupon cash flows
cash_flows = create_cashflow_dataframe(coupon_flows, unique_list)
# Dataframe with bond notional cash flows
notional_cash_flows = create_cashflow_dataframe(notional_flows,␣
 ↪unique_terminal_list)
```

Cash flow dataframe with coupon amounts and dates:

---

[19]:
```
display(cash_flows.head())
```

```
              2023-06-30  2023-09-01  2024-01-28  2024-03-17  2024-03-21  \
BE6285455497       0.000       0.000       0.000         2.0       0.000
FR001400CFW8       0.000       3.625       0.000         0.0       0.000
FR0013324357       0.000       0.000       0.000         0.0       1.375
FR0000471930       0.000       0.000       8.125         0.0       0.000
FR001400DNF5       5.375       0.000       0.000         0.0       0.000

              2024-06-30  2024-09-01  2025-01-28  2025-03-17  2025-03-21  …  \
BE6285455497       0.000       0.000       0.000         2.0       0.000  …
FR001400CFW8       0.000       3.625       0.000         0.0       0.000  …
FR0013324357       0.000       0.000       0.000         0.0       1.375  …
FR0000471930       0.000       0.000       8.125         0.0       0.000  …
FR001400DNF5       5.375       0.000       0.000         0.0       0.000  …

              2033-06-30  2034-06-30  2035-06-30  2036-06-30  2037-06-30  \
BE6285455497       0.000       0.000       0.000       0.000       0.000
FR001400CFW8       0.000       0.000       0.000       0.000       0.000
FR0013324357       0.000       0.000       0.000       0.000       0.000
FR0000471930       0.000       0.000       0.000       0.000       0.000
FR001400DNF5       5.375       5.375       5.375       5.375       5.375
```

	2038-06-30	2039-06-30	2040-06-30	2041-06-30	2042-06-30
BE6285455497	0.000	0.000	0.000	0.000	0.000
FR001400CFW8	0.000	0.000	0.000	0.000	0.000
FR0013324357	0.000	0.000	0.000	0.000	0.000
FR0000471930	0.000	0.000	0.000	0.000	0.000
FR001400DNF5	5.375	5.375	5.375	5.375	5.375

[5 rows x 49 columns]

Cash flow dataframe with notional amount payments and dates:

---

[20]:
```
display(notional_cash_flows.head())
```

	2028-03-17	2029-09-01	2030-03-21	2033-01-28	2042-06-30
BE6285455497	100.0	0.0	0.0	0.0	0.0
FR001400CFW8	0.0	100.0	0.0	0.0	0.0
FR0013324357	0.0	0.0	100.0	0.0	0.0
FR0000471930	0.0	0.0	0.0	100.0	0.0
FR001400DNF5	0.0	0.0	0.0	0.0	100.0

1.6 5. Step 4 in numbers: pricing off the risk-free curve

This section prices the cash flows from section 4 off the risk-free curve (Step 4), first on its own and then with the input z-spread added.

1.6.1 The risk-free curve

import_SWEiopa() reads the necessary data about the current yield curve. The risk free term structure it produces is what every bond cash flow is discounted against, before the bond specific z-spread is added on top. Inside OSEM, the parameters related to the yield curve are saved in the Curves object.

[21]:
```
[maturities_country, curve_country, extra_param, Qb] = import_SWEiopa(settings.
 ↪EIOPA_param_file,
                                                                        ␣
 ↪settings.EIOPA_curves_file, settings.country)
# Curves object with information about term structure
curves = Curves(extra_param["UFR"] / 100, settings.precision, settings.tau,␣
 ↪settings.modelling_date,
                settings.country)
```

[22]:
```
ufr = extra_param["UFR"]/100 # ultimate forward rate
precision = float(settings.precision) # Numeric precision of the optimisation
```

```
# Targeted distance between the extrapolated curve and the ufr at the convergence point
tau = float(settings.tau) # 1 basis point
```

[23]:
```
curves.set_observed_term_structure(maturity_vec=curve_country.index.to_numpy(dtype=float), yield_vec=curve_country.values)
curves.calc_fwd_rates()
# The +1 matches main.py: the last projection period still has to be priced, so the
# projected curve has to reach one year past the end of the modelling window.
curves.project_forward_rate(settings.n_proj_years+1)
curves.calibrate_projected(settings.n_proj_years+1, 0.05, 0.5, 1000)
```

1.6.2 Pricing state at the modelling date

`init_bond_portfolio_to_dataframe` sets up the three DataFrames the pricing works on, each with one column for the modelling date: the market price of each bond, its z-spread and the units held.

[24]:
```
[market_price_df, zspread_df, units_df] = bond_portfolio.init_bond_portfolio_to_dataframe(settings.modelling_date)
```

At this point `zspread_df` holds the `Z_Spread` read from the portfolio file, the extra yield over the risk-free curve for the bond's credit and liquidity risk. The calibration in sections 6 and 7 overwrites it:

---

[25]:
```
display(zspread_df)
```

	2023-04-29
BE6285455497	0.0052
FR001400CFW8	0.0055
FR0013324357	0.0048
FR0000471930	0.0077
FR001400DNF5	0.0046

All pricing in this notebook happens at the modelling date, so the projection year is 0:

[26]:
```
proj_period = 0
```

1.6.3 Risk-free price: the gap the z-spread has to close

Pricing the cash flows at $z = 0$ gives MV_{rf} of Step 4, the value of the bond if it carried no credit or liquidity risk at all. The risk free price is what the cash flows are worth on the EIOPA curve alone, the market price is what the bond actually trades at, and the z-spread is the constant addition to the curve that closes the difference.

---

[27]:
```
risk_free_price = {}
for asset_id, bond in bond_portfolio.bonds.items():
    risk_free_price[asset_id] = bond.price_bond(cash_flows.loc[asset_id],
                                                notional_cash_flows.
 ↪loc[asset_id],
                                                settings.modelling_date,
                                                proj_period,
                                                curves,
                                                0.0)

price_comparison = pd.DataFrame({
    "Risk free price": pd.Series(risk_free_price),
    "Market price": pd.Series({aid: b.market_price
                               for aid, b in bond_portfolio.bonds.items()}),
})
price_comparison["Gap to close"] = (price_comparison["Risk free price"]
                                    - price_comparison["Market price"])

display(price_comparison)
```

	Risk free price	Market price	Gap to close
BE6285455497	93.660532	93.4649	0.195632
FR001400CFW8	103.078708	99.4582	3.620508
FR0013324357	87.278258	86.4098	0.868458
FR0000471930	139.140211	132.6311	6.509111
FR001400DNF5	128.745950	108.2142	20.531750

1.6.4 Price at the input z-spread

The same cash flows discounted at the risk free curve **plus the input `Z_Spread`** held in `zspread_df`. These prices do not reproduce the observed `Market_Price` either; closing that gap is what the calibration in the next section does.

Note that `main.py` has no counterpart to this step. There the bond price at the modelling date is the `Market_Price` from the input file, and the first discounted cash flow price is produced inside the main loop.

[28]:
```
market_price_df = bond_portfolio.price_bond_portfolio(cash_flows,␣
 ↪notional_cash_flows, proj_period, curves, zspread_df,␣
 ↪market_price_df,settings.modelling_date)
```

---

[29]:
```
price_comparison.insert(1, "Price at input z-spread", market_price_df.iloc[:,␣
 ↪0])

display(price_comparison)
```

```
              Risk free price  Price at input z-spread  Market price  \
BE6285455497        93.660532                91.488029       93.4649
FR001400CFW8       103.078708               100.039972       99.4582
FR0013324357        87.278258                84.658413       86.4098
FR0000471930       139.140211               131.746371      132.6311
FR001400DNF5       128.745950               121.761314      108.2142

              Gap to close
BE6285455497      0.195632
FR001400CFW8      3.620508
FR0013324357      0.868458
FR0000471930      6.509111
FR001400DNF5     20.531750
```

1.7 6. Step 5 in numbers: calibrating one bond

Step 5 for bond BE6285455497 (Anheuser-Busch InBev, 2% due 2028): the bisection searches the bracket $[-0.2, 0.2]$ for the spread that, added on top of the risk free term structure, makes the discounted cash flows equal the market price.

[30]:
```
calibrated_spread = bond_portfolio.bonds["BE6285455497"].
↪bisection_spread(x_start=-0.2
                                , x_end=0.2
                                , modelling_date=settings. modelling_date
                                , end_date=settings.end_date
                                , proj_period=proj_period
                                , curves=curves
                                , precision= 0.00000001
                                , max_iter=100000)
```

The market value calculated using the discounted cash flow method using the calibrated zspread is:

[31]:
```
calibrated_bond = bond_portfolio.bonds["BE6285455497"].price_bond(cash_flows.
↪loc["BE6285455497"],notional_cash_flows.loc["BE6285455497"],settings.
↪modelling_date, proj_period,curves,calibrated_spread)
```

---

[32]:
```
print(calibrated_bond)
```

```
93.46490215037936
```

1.8 7. Step 6 in numbers: calibrating the whole portfolio

`calibrate_bond_portfolio` repeats the bisection for every bond (Step 6) and writes the results into `zspread_df`:

[33]:
```
zspread_df=bond_portfolio.calibrate_bond_portfolio(zspread_df, settings,␣
↪proj_period, curves)
```

---

[34]:
```
display(zspread_df)
```

	2023-04-29
BE6285455497	0.000462
FR001400CFW8	0.006577
FR0013324357	0.001572
FR0000471930	0.006749
FR001400DNF5	0.014664

Repricing the portfolio with the calibrated spreads reproduces the observed market prices. The residual is the price cost of stopping the bisection once the spread bracket is narrower than `precision`, as described in Step 5:

---

[35]:
```
calibrated_price_df = bond_portfolio.price_bond_portfolio(cash_flows,
                                                          notional_cash_flows,
                                                          proj_period,
                                                          curves,
                                                          zspread_df,
                                                          market_price_df.copy(),
                                                          settings.
↪modelling_date)

calibration_check = pd.DataFrame({
    "Calibrated z-spread": zspread_df.iloc[:, 0],
    "Repriced value": calibrated_price_df.iloc[:, 0],
    "Market price": pd.Series({aid: bond.market_price
                               for aid, bond in bond_portfolio.bonds.items()}),
})
calibration_check["Residual"] = (calibration_check["Repriced value"]
                                 - calibration_check["Market price"])

display(calibration_check)
```

	Calibrated z-spread	Repriced value	Market price	Residual
BE6285455497	0.000462	93.464902	93.4649	2.150379e-06
FR001400CFW8	0.006577	99.458197	99.4582	-2.511242e-06
FR0013324357	0.001572	86.409802	86.4098	2.366104e-06
FR0000471930	0.006749	132.631099	132.6311	-9.114772e-07
FR001400DNF5	0.014664	108.214198	108.2142	-2.024207e-06

1.9 8. Appendix: input data

The input files this notebook reads, all in `notebooks/data/Input`. Their locations come from the `[INPUT]` section of `notebooks/data/notebooks.ini`, except the EIOPA files, whose paths are set in `Parameters.csv`. Dates are written as `dd/mm/yyyy`.

1.9.1 Parameters.csv

The run parameters, one `Parameter`, `Value` pair per row. Loaded into `Settings` by `get_settings`.

Parameter	Example	Meaning
`EIOPA_param_file`	`notebooks/data/Input/Param_no_VA.csv`	Relative path of the EIOPA parameter file
`EIOPA_curves_file`	`notebooks/data/Input/Curves_no_VA.csv`	Relative path of the EIOPA spot curve file
`country`	`Slovenia`	Country whose curve is used; must match a column name in both EIOPA files
`n_proj_years`	`50`	Length of the run in years. The end of the modelling window E is the modelling date plus this many years
`Precision`	`1E-10`	Precision of the Smith-Wilson calibration in `Curves`. The bond z-spread calibration does not use it: `calibrate_bond_portfolio` hardcodes `1e-8`
`Tau`	`0.0001`	Accepted gap between the extrapolated forward rate and the UFR at the convergence point (1 bp)
`compounding`	`-1`	Compounding convention. Read into `Settings` but not used yet; discounting is always annual, $(1+y)^{-t}$
`Modelling_Date`	`29/04/2023`	Start date of the run, MD
`liability_mode`	`cashflow`	`cashflow` or `unit_linked`; selects the liability model of the full run. Not used in this notebook
`random_seed`	`42`	Seed for the random draws in `unit_linked` mode. Not used in this notebook

1.9.2 EIOPA RFR files

Two files derived from EIOPA's monthly risk-free rate (RFR) publication. `import_SWEiopa` reads the columns of the chosen `country` from each. The `no_VA` files hold the curves without the volatility adjustment.

- **`Curves_no_VA.csv`**: spot rates, one row per maturity (1 to 150 years) and one column per country, as decimals (`0.03472` is 3.472%).
- **`Param_no_VA.csv`**: the parameters EIOPA used to build the curves, in two columns per country (`<Country>_Maturities`, `<Country>_Values`). The first six rows are scalar parameters: `Coupon_freq`, `LLP` (last liquid point), `Convergence` (convergence period), `UFR` (ultimate forward rate, in percent, e.g. `3.45`), `alpha` (speed of convergence to the UFR) and `CRA` (credit risk adjustment, in bp). The rows after them hold the Smith-Wilson calibration vector by maturity.

Together they give the risk free term structure at the modelling date, from which `Curves` projects its evolution over the run.

1.9.3 Corporate_Bond_Portfolio.csv

The corporate bond portfolio, one row per bond position. Loaded by `get_corporate_bonds`, one `CorpBond` per row. Section 3 shows which of these fields enter the price.

Column	Example	Meaning
`Asset_ID`	`BE6285455497`	Unique id of the position, read as text: the bond's ISIN
`Asset_Type`	`Corporate_Bond`	Asset type; `get_corporate_bonds` raises if it is not `Corporate_Bond`
`NACE`	`C11.0.5`	NACE code of the issuer (*nomenclature statistique des activités économiques dans la Communauté européenne*)
`Issue_Date`	`17/03/2016`	Issue date I; anchors the coupon grid. For a bond whose first coupon period was short, this is the start of that period rather than the day the bond was issued, so that the coupons fall on the anniversaries of the maturity date
`Maturity_Date`	`17/03/2028`	Maturity date M; the notional is repaid on it
`Notional_Amount`	`100`	Notional amount N

Column	Example	Meaning
`Coupon_Rate`	`0.02`	Coupon c paid on **each** coupon date, as a fraction of the notional. Not annualised: see the note on f under Notation
`Spread_Country`	`0`	Country spread; not used in bond pricing
`Spread_Sector`	`0`	Sector spread; not used in bond pricing
`Z_Spread`	`0.0052`	Starting z-spread; replaced by the calibrated value
`Spread_Stress`	`0`	Spread added in a stress scenario; not used in bond pricing
`Frequency`	`1`	Coupon payments per year f: 1, 2, 3, 4 or 12
`Recovery_Rate`	`0.4`	Fraction of the notional recovered on default; not used yet
`Default_Probability`	`0.00077`	Annual probability of default; not used yet
`Units`	`40.9441`	Number of bonds held
`Market_Price`	`95.7808`	Price of one bond at the modelling date, on the same scale as `Notional_Amount`; the calibration target MV
`Issuer`	`ANHEUSER-BUSCH INBEV NV`	Name of the issuer; for reference only, `get_corporate_bonds` does not read it

1.9.4 Sector_Spread.csv

Sector spreads by NACE code. In this proof of concept the bond spread comes from `Corporate_Bond_Portfolio.csv`, so this file is loaded in section 3 for reference only.

Column	Example	Meaning
`NACE`	`A1`	NACE code, at section, division or group level
`NACE code text`	`Crop and animal production, hunting and related service activities`	Description of the NACE code
`sSpread`	`0.01`	Sector specific spread over the risk free rate